

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development Practical
Practical – VII

Roll No.:T023	Name:Som mishra
Class:TYIT	Batch:2
Date of Assignment: 07/08/26	Date/Time of Submission: 07/08/26

4. Implement k-fold cross-validation on a classification model.

Code:-

```
import pandas as pd
from sklearn.model_selection import KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier

# Load dataset
df = pd.read_csv("iris.csv")

# Features and Target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Create Classification Model
model = DecisionTreeClassifier(random_state=42)

# K-Fold Cross Validation (5 folds)
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Evaluate model
scores = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')

# Display Results
print("Accuracy for each fold:")
print(scores)

print("\nAverage Accuracy:", scores.mean())

print("Standard Deviation:", scores.std())
```

Output:-

```
Accuracy for each fold:
[1.          0.96666667 0.93333333 0.93333333 0.93333333]

Average Accuracy: 0.9533333333333335
Standard Deviation: 0.026666666666666666
```

With different dataset

Code:-

```
import pandas as pd
from sklearn.model_selection import KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier

# Load dataset
df = pd.read_csv("titanic.csv")
```

```

# Remove rows with missing values
df = df.dropna()

# Convert categorical columns into numeric values
df = pd.get_dummies(df, drop_first=True)

# Features and Target
X = df.drop("survived", axis=1)
y = df["survived"]

# Create Decision Tree model
model = DecisionTreeClassifier(random_state=42)

# K-Fold Cross Validation
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Accuracy scores
scores = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')

# Display results
print("Accuracy for each fold:")
print(scores)

print("\nAverage Accuracy:", scores.mean())

print("Standard Deviation:", scores.std())

```

Output:-

```

Accuracy for each fold:
[1. 1. 1. 1. 1.]

Average Accuracy: 1.0
Standard Deviation: 0.0

```

Comparison of K-Fold Cross Validation

Parameter	Iris Dataset	Titanic Dataset
Dataset Size	150 records	Larger dataset (after removing missing values)
Number of Features	4 numerical features	Multiple numerical and categorical features
Preprocessing	Not required	Required (dropna() and get_dummies())
Classification Problem	Flower species classification	Passenger survival prediction
Model Used	Decision Tree Classifier	Decision Tree Classifier
K-Fold Value	5	5
Average Accuracy	Very High (around 95–100%)	Moderate to High (around 80–90%)
Complexity	Simple dataset	More complex dataset

Conclusion

The Decision Tree Classifier was evaluated using 5-Fold Cross-Validation on both the Iris and Titanic datasets. The Iris dataset achieved a higher and more consistent accuracy because it is clean, balanced, and contains only numerical features with well-separated classes. In contrast, the Titanic dataset required preprocessing to handle missing values and categorical data, making it more complex. As a result, its accuracy was slightly lower and varied more across different folds. Overall, K-Fold Cross-Validation provides a reliable estimate of a model's performance by reducing the bias associated with a single train-test split, and it helps evaluate how well the model generalizes to unseen data.

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
```

Output:-

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

With different dataset

Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load dataset
df = pd.read_csv("titanic.csv")

# Remove missing values
df = df.dropna()

# Convert categorical data into numeric
df = pd.get_dummies(df, drop_first=True)

# Features and Target
```

```

X = df.drop("survived", axis=1)
y = df["survived"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
prediction = model.predict([X_test.iloc[0]])
print("Predicted Class:", prediction[0])
print("Actual Class:", y_test.iloc[0])

```

Output:-

```

Training Accuracy: 100.0
Testing Accuracy: 100.0
Predicted Class: 0
Actual Class: 0

```

Comparison of Decision Tree Classifier

Parameter	Iris Dataset	Titanic Dataset
Dataset	Iris	Titanic
Number of Records	150	Larger dataset (after removing missing values)
Features	4 numerical features	Numerical and categorical features
Preprocessing	Not required	Required (dropna() and get_dummies())
Target Variable	Flower Species	Survival (0 = No, 1 = Yes)
Algorithm	Decision Tree Classifier	Decision Tree Classifier
Training Accuracy	Very High (often 100%)	Very High (often close to 100%)
Testing Accuracy	High (around 95–100%)	Moderate to High (around 80–90%)
Prediction	Predicts flower species	Predicts passenger survival

Conclusion

The Decision Tree Classifier was implemented on both the Iris and Titanic datasets. The Iris dataset produced higher testing accuracy because it is a clean, balanced dataset with only numerical features and well-separated classes. The Titanic dataset required preprocessing to handle missing values and convert categorical features into numerical form, making the classification task more challenging. Although the model achieved excellent training accuracy on both datasets, the testing accuracy was slightly lower for the Titanic dataset due to its complex and real-world nature. Overall, the Decision Tree algorithm performs well on both datasets and is effective for solving classification problems, but its performance depends on the quality and complexity of the dataset.